

LLM-BugBounty-Automation

전체 시스템 설계 및 단계별 구현 로드맵

v1.0 설계안

2026-09-21

문서 목적

허가된 LLM·AI Agent·RAG·웹 기반 AI 서비스를 대상으로 정찰, 테스트 생성, 실행, 판정, 재현, 증거화, 중복 제거, 버그바운티 보고서 작성까지 일관되게 자동화하는 플랫폼을 설계한다. 각 구현 단계가 끝날 때마다 테스트 후 GitHub commit/push 를 수행하고, 의미 있는 마일스톤에는 태그를 생성하는 개발 방식을 기본 원칙으로 한다.

Scope first · Evidence driven · Reproducible · Tool-agnostic

1. 프로젝트 개요

이 프로젝트의 목표는 “LLM에게 해킹을 시켜 보는 스크립트”가 아니라, 허가된 AI 시스템을 반복 가능하고 증거 중심으로 평가하는 보안 테스트 플랫폼을 만드는 것이다. 핵심은 외부 스캐너의 결과를 그대로 믿지 않고, 모든 결과를 표준 포맷으로 정규화한 뒤 다중 Judge, Control Test, 재현 테스트를 거쳐 실제 Finding으로 승격하는 데 있다.

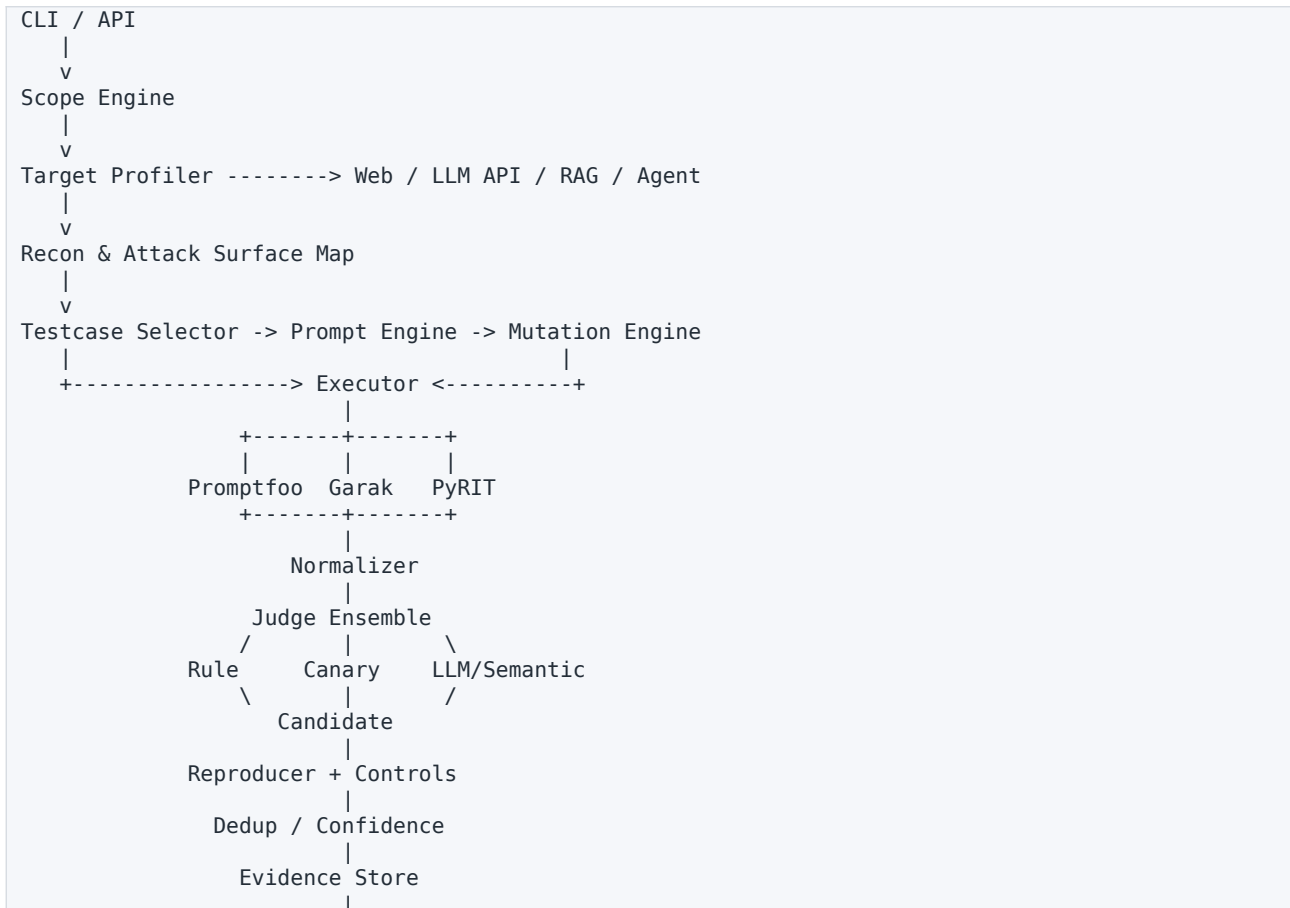
1.1 핵심 목표

- 웹 기반 AI 서비스, LLM API, RAG, AI Agent를 하나의 Target 추상화로 다룬다.
- Recon → 공격 표면 분류 → LLM Red Team → 판정 → 재현 → 보고서 생성을 하나의 파이프라인으로 연결한다.
- Promptfoo, Garak, PyRIT, Nuclei 등 외부 도구는 Adapter 뒤에 배치하여 교체 가능한 Tool Layer로 사용한다.
- Prompt와 Mutation의 lineage를 보존해 어떤 공격 전략이 성공했는지 분석 가능하게 만든다.
- 모든 요청/응답/판정/재현 결과를 Evidence로 저장해 버그바운티 제출 자료로 활용할 수 있게 한다.
- 실제 실행 전에 Scope Engine이 허용 범위, 속도 제한, 동시성 제한을 강제한다.

1.2 v1 완료 기준

영역	v1 완료 기준
안전성	모든 실행이 Scope 검증을 통과해야 하며 기본 rate limit/concurrency 정책이 존재
LLM 테스트	YAML Testcase → Executor → Judge → Reproducer 흐름이 동작
외부 도구	Promptfoo/Garak/PyRIT 및 핵심 Recon·Web 도구가 Adapter로 통합
데이터	Run/Request/Response/Finding/Evidence/Reproduction이 DB와 파일에 저장
품질	False Positive 감소를 위한 Control Test와 재현률 계산 지원
출력	Markdown/JSON 버그바운티 보고서 자동 생성
개발 프로세스	각 Phase 종료 시 test → commit → push, 마일스톤 태그 운영

2. 전체 아키텍처



2.1 설계 원칙

- Scope-first: 모든 네트워크 작업은 Scope Engine 앞을 통과하지 못한다.
- Adapter-first: 외부 도구의 출력 포맷을 코어 로직이 직접 알지 않는다.
- Evidence-first: Finding 보다 먼저 원시 요청/응답과 판정 근거를 보존한다.
- Reproduction-first: 단발 성공은 Candidate 일 뿐 Confirmed 가 아니다.
- Deterministic where possible: Regex, Canary, Control Test 를 LLM Judge 보다 우선한다.
- Replaceable components: Scanner, Target Provider, Judge, Storage 를 독립 모듈로 설계한다.

3. 외부 도구 선정 및 역할

도구	분류	역할	통합 전략
Promptfoo	LLM Red Team	정형 테스트, eval, red-team suite	JSON 결과 → PromptfooAdapter → FindingCandidate
Microsoft PyRIT	LLM Red Team	Adaptive / Multi-turn 공격, 변형	후속 심층 검증 Agent 로 사용
NVIDIA Garak	LLM Scanner	다양한 probe/detector 기반 취약점 탐색	JSONL → GarakAdapter
Subfinder	Recon	Passive subdomain enumeration	JSONL → Asset
httpx	Recon	Live HTTP 확인, 메타데이터/기술 탐지	JSONL → Endpoint
Katana	Discovery	URL/API/JS crawling	JSONL → DiscoveredEndpoint
ffuf	Discovery	숨은 경로/파라미터 fuzzing	JSON → DiscoveredEndpoint
Nuclei	Web Scanner	Web/API/설정 취약점 검사	JSONL → FindingCandidate
Dalfox	XSS Scanner	XSS 및 파라미터 분석, MCP/REST 활용 가능	JSONL → FindingCandidate
TruffleHog	Secret Scanner	API Key/Token/Credential 탐지	JSON → SecretFinding

권장 원칙

올인원 자동화 프로젝트 여러 개를 한꺼번에 합치지 않는다. 검증된 단일 목적 도구를 Tool Layer 로 배치하고, 오케스트레이션·판정·재현·증거화는 자체 코드가 담당하도록 한다.

4. Repository 구조

```

LLM-BugBounty-Automation/
├── README.md / LICENSE / pyproject.toml / .env.example
├── main.py / cli.py
├── config/
├── core/
├── scope/
├── targets/
├── adapters/
│   ├── llm/
│   ├── recon/
│   ├── discovery/
│   ├── scanner/
│   └── secrets/
├── recon/
├── attacks/
├── prompts/
├── testcase/
├── executor/
├── judges/
├── reproduction/
├── findings/
├── storage/
└── reporting/
# pipeline, model, scope, tools, logging
# orchestrator, scheduler, context, events
# validator, allowlist, denylist, rate limit
# LLM/API/WebChat/Agent target adapters
# external tools adapters
# promptfoo, garak, pyrit
# subfinder, httpx
# katana, ffuf
# nuclei, dalfox
# trufflehog
# attack surface mapping/classification
# planner, generator, mutation, lineage
# seed/mutation/judge/report prompts
# YAML schemas and suites
# async runner, sessions, retry, throttling
# rule, regex, canary, semantic, LLM, ensemble
# repeat/control/statistics
# normalize/dedup/confidence/severity
# DB/evidence/artifacts
# Markdown/JSON report generator
  
```

```

├── knowledge/      # techniques/vulnerabilities/sources
├── tests/          # unit/integration/fixtures
├── runs/
├── evidence/
└── reports/

```

4.1 디렉터리 책임

모듈	책임
core	파이프라인 상태, 실행 순서, 이벤트, 공통 컨텍스트 관리
scope	대상 허용 여부와 네트워크 실행 정책을 중앙에서 강제
targets	대상별 호출 방식을 공통 인터페이스로 추상화
adapters	외부 CLI/서비스 결과를 내부 표준 모델로 변환
attacks	테스트 선택, Prompt 생성, Mutation, lineage 기록
judges	응답을 여러 신호로 평가하고 confidence 산출
reproduction	반복/대조 실험으로 재현성 검증
findings	후보 정규화, 중복 제거, 심각도/신뢰도 계산
storage	DB 및 파일 Evidence 보존
reporting	확정 Finding 을 제출 가능한 보고서로 변환

5. 핵심 데이터 모델

코어 로직은 외부 도구별 포맷이 아니라 내부 표준 객체만 사용해야 한다. 최소 객체는 Target, Run, Testcase, Prompt, Mutation, Request, Response, Judgement, Finding, Reproduction, Evidence, Report 다.

```

Finding
- id
- run_id / target_id
- category / title
- source_tool
- endpoint
- testcase_id / mutation_id
- request_ref / response_ref
- confidence
- severity
- status: candidate | confirmed | rejected | unstable
- reproduction_rate
- evidence_refs[]
- root_cause_key
- created_at

```

5.1 Normalized Result 계약

모든 Adapter 는 최소한 아래 형태를 반환하도록 고정한다. 이 계약이 외부 도구 교체 가능성을 보장한다.

```

NormalizedResult {
  run_id,
  target_id,
  source: { tool, version },
  category,
  title,
  endpoint,
  raw_artifact_ref,
  request_ref,
  response_ref,
  detector_score,
  metadata
}

```

6. 저장소 및 DB 설계

MVP 는 SQLite 로 시작하고, 동시 Worker 나 다중 사용자 Dashboard 가 필요해질 때 PostgreSQL 로 이전한다. 원시 Evidence 파일과 구조화 DB 를 분리하여 DB 가 커지는 것을 방지한다.

테이블	주요 필드/역할
-----	----------

targets	target_id, type, base_url, provider, scope_profile
runs	run_id, target_id, profile, status, started_at, finished_at
assets/endpoints	domain, url, method, source, classifier_type
testcases	testcase_id, category, version, source
prompts/mutations	prompt_hash, parent_id, strategy, generation
requests/responses	artifact path, HTTP/LLM metadata, latency, tokens
judgements	judge_type, score, reason/evidence ref
findings	category, status, confidence, severity, root_cause_key
reproductions	attempts, successes, controls, success_rate
evidence	finding_id, artifact_type, path/hash
reports	format, path, generated_at, source_findings

7. Scope Engine

Scope Engine 은 옵션이 아니라 시스템의 실행 게이트다. 모든 HTTP 요청, 외부 스캐너 실행, 브라우저/Agent 호출은 Scope 검증 결과를 전달받아야 한다.

```
program: local-test
allowed:
  domains:
    - localhost
    - lab.example.com
  url_patterns:
    - https://lab.example.com/api/*
denied:
  domains:
    - production.example.com
limits:
  requests_per_second: 2
  concurrency: 3
  max_requests_per_run: 5000
```

- Target URL/domain/IP allowlist 및 명시적 denylist 를 지원한다.
- 서브도메인 범위 포함 여부를 설정으로 분리한다.
- Tool 실행 전 scope snapshot 을 Evidence 에 저장한다.
- rate limit, concurrency, request budget 을 중앙에서 강제한다.
- out-of-scope redirect 발견 시 자동 중지 또는 기록-only 정책을 적용한다.

8. Target Adapter

```
class TargetAdapter:
    async def healthcheck(self): ...
    async def metadata(self): ...
    async def send(self, input, session=None): ...
    async def reset_session(self): ...
```

초기 Target 타입은 OpenAI-compatible API, Custom HTTP API, Web Chat Wrapper, Fake LLM 테스트 타깃으로 제한한다. RAG/Agent 는 공통 인터페이스가 안정화된 후 확장한다.

9. Testcase 및 Prompt Engine

```
id: LLM-PI-001
name: Basic Instruction Override
category: prompt_injection
target_types: [llm, agent]
prompt: |
  {{PAYLOAD}}
mutation:
  enabled: true
judges: [rule, semantic, llm]
reproduce:
  attempts: 5
  threshold: 3
severity:
```

base: medium

프롬프트를 코드에 하드코딩하지 않고 YAML/JSON 버전 관리 대상으로 둔다. 동일 Testcase 에서 여러 Mutation 을 생성할 수 있어야 하며, 각 Prompt 는 content hash 로 추적한다.

9.1 초기 Test Suite

카테고리	예시 테스트 목적
prompt_injection	상위 지시 무시 또는 정책 우회 여부
system_prompt_leak	숨은 지시/Canary 정보 노출 여부
indirect_injection	외부 문서/RAG 컨텍스트를 통한 지시 삽입
rag_security	검색 문서 오염, source attribution 오류, 권한 경계
tool_abuse	허용되지 않은 Tool 호출/인자 조작
agent_security	과도한 권한, 작업 범위 이탈, 연쇄 행동
data_leak	민감정보/다른 컨텍스트 정보 노출
multi_turn	세션 누적을 이용한 정책/상태 교란

9.2 교재 기반 Seed 활용

업로드된 2025 버그헌팅 중급과정 교재의 ChatGPT 버그헌팅 구간(386~395 쪽)은 “정답 공격법”이 아니라 Seed/Mutation 아이디어로 사용한다. step-by-step, 반복 질문, 반심리학, 소유권 주장, 도전형 요청은 social/reasoning mutation 으로 분류하고, 소스 분석·자동화 스크립트·Burp 확장·보고서 작성 항목은 각각 Code Analysis, PoC/Automation, Tool Integration, Report Generator 설계에 반영한다.

10. Mutation Engine

Mutation Engine 은 무작위 Prompt 변형기가 아니라 성공·실패 계보를 추적하는 실험 엔진이어야 한다.

```
Seed Prompt
├── paraphrase
├── translation
├── roleplay
├── JSON/XML/Markdown wrapping
├── encoding / unicode variation
├── multi-turn split
├── ownership claim
├── reverse psychology / challenge
└── model-generated mutation
```

- mutation_id, parent_mutation_id, strategy, generation 을 저장한다.
- 동일 Prompt 중복 생성을 hash 로 제거한다.
- 전략별 성공률과 Target 별 성공률을 집계할 수 있게 한다.
- 성공한 Mutation 을 바로 “취약점”으로 처리하지 않고 Reproducer 로 전달한다.

11. Executor 및 세션 관리

- asyncio + httpx 기반 비동기 실행
- timeout, retry, exponential backoff
- global 및 target 별 concurrency 제한
- multi-turn session 유지/초기화
- request/response 원문 저장
- latency, token/cost metadata 저장
- Run 중단 후 resume 를 위한 체크포인트

12. Judge Ensemble

LLM 보안 자동화에서 가장 위험한 부분은 False Positive 다. 따라서 LLM Judge 하나가 아니라 여러 독립 신호를 합친다.

Judge	목적	신뢰도 성격
-------	----	--------

Rule	정확한 문자열/구조/상태 변화 검증	높음 - 결정적 조건
Regex	키/토큰/정해진 패턴 검출	높음 - 패턴 의존
Canary	사전에 삽입한 비밀 marker 유출 확인	매우 높음
Semantic	의미상 지시 위반/정보 유출 유사도 평가	중간
LLM Judge	복잡한 행동·문맥 기반 결과 평가	보조 신호
Ensemble	각 점수/증거를 조합해 confidence 산출	최종

```

Example
rule      = 0.40
canary    = 1.00
semantic  = 0.85
llm       = 0.92
-----
final confidence ≈ 0.93

```

13. 재현 및 Control Test

```

Initial Hit
|
v
Negative Control ----> baseline behavior
|
v
Repeat N times
|
v
Variation Test
|
v
Cross-tool validation
|
+--> confirmed / unstable / rejected

```

- 기본 attempts=5 또는 10, threshold 를 Testcase 별로 설정한다.
- Control Prompt 에서는 동일 민감 정보가 정상적으로 노출되는지 확인한다.
- temperature/model/session 조건을 Evidence 에 기록한다.
- reproduction_rate 를 Finding 핵심 필드로 둔다.

14. 중복 제거 및 Root Cause 묶음

동일 취약점이 Promptfoo, Garak, PyRIT, 자체 Mutation 에서 각각 발견될 수 있으므로 도구별 Finding 을 그대로 보고하면 품질이 떨어진다.

1. category + endpoint + normalized behavior 를 이용한 exact/heuristic dedup
2. 응답 signature 와 핵심 증거 문자열 비교
3. 제목/설명 semantic similarity 비교
4. root_cause_key 를 생성해 Cluster 단위로 보고
5. v2 에서 pgvector 기반 embedding clustering 으로 확장

15. Evidence 및 Reporting

```

runs/RUN-.../
├── metadata.json
├── scope.json
├── recon/
├── requests/
├── responses/
├── prompts/
├── mutations/
├── findings/
├── report/

evidence/FINDING-001/
├── testcase.yaml
├── request.json
├── response.json

```

```
├─ judge.json
├─ reproduction.json
└─ metadata.json
```

보고서는 Title, Summary, Affected Component, Prerequisites, Steps to Reproduce, Input/Prompt, Observed Behavior, Expected Behavior, Security Impact, Reproduction Rate, Evidence, Suggested Mitigation 을 기본 항목으로 한다.

16. 실행 프로필

프로필	목적	구성
quick	빠른 상태 확인	Scope → Target → 핵심 Testcase → Rule/Canary Judge
llm	LLM/API 집중	Promptfoo/Garak → Judge → Reproducer
agent	Tool/Agent 권한 집중	Agent suites → multi-turn → PyRIT adaptive
rag	RAG/문서 주입 집중	indirect injection → attribution/data leakage tests
web	웹 공격면 집중	Recon → Nuclei/Dalfox/TruffleHog
full	전체 평가	Recon + LLM + Web + 재현 + 보고서

17. 단계별 구현 로드맵

GitHub 운영 원칙

각 Phase 가 끝날 때 반드시 자동/수동 테스트를 통과시키고 `git add → commit → push`를 수행하는 것을 권장한다. 의미 있는 마일스톤은 태그를 생성한다. 기능이 완성되지 않은 큰 덩어리를 오래 로컬에만 보관하지 않는다.

Phase 0 - Repository 초기화

항목	내용
구현 내용	README, LICENSE, .gitignore, pyproject.toml, .env.example, 기본 디렉터리 생성
완료 조건	저장소 clone 후 기본 CLI 가 실행되고 빈 pytest 가 정상 동작
권장 Commit	chore: initialize LLM bug bounty automation project
권장 Tag	v0.0.1

GitHub 체크포인트: Phase 종료 직후 테스트 결과를 확인한 다음 commit/push 한다. main 에 병합되는 마일스톤이라면 태그도 push 하는 것을 권장한다.

Phase 1 - Core Data Model

항목	내용
구현 내용	Target, Run, Request, Response, Finding, Evidence 모델과 SQLite schema
완료 조건	객체 생성/직렬화/DB 저장/조회 unit test 통과
권장 Commit	feat: add core domain models and storage schema
권장 Tag	v0.1.0-core

GitHub 체크포인트: Phase 종료 직후 테스트 결과를 확인한 다음 commit/push 한다. main 에 병합되는 마일스톤이라면 태그도 push 하는 것을 권장한다.

Phase 2 - Scope Engine

항목	내용
구현 내용	allow/deny, URL/domain validation, rate limit, concurrency, request budget
완료 조건	허용 대상은 통과하고 금지 대상은 실행 전 차단

권장 Commit	feat: implement target scope and execution policy
권장 Tag	v0.2.0-scope

GitHub 체크포인트: Phase 종료 직후 테스트 결과를 확인한 다음 commit/push 한다. main 에 병합되는 마일스톤이라면 태그도 push 하는 것을 권장한다.

Phase 3 - Target Adapter

항목	내용
구현 내용	BaseTarget, OpenAICompatible, CustomHTTP, FakeLLM
완료 조건	FakeLLM 대상으로 healthcheck/send/session 테스트 통과
권장 Commit	feat: add target adapter architecture
권장 Tag	v0.3.0-targets

GitHub 체크포인트: Phase 종료 직후 테스트 결과를 확인한 다음 commit/push 한다. main 에 병합되는 마일스톤이라면 태그도 push 하는 것을 권장한다.

Phase 4 - Testcase / Prompt Engine

항목	내용
구현 내용	YAML schema, loader, 변수 치환, version/hash, 교재 Seed 초기 등록
완료 조건	YAML validation 과 Prompt 렌더링 테스트 통과
권장 Commit	feat: implement testcase and prompt seed engine
권장 Tag	v0.4.0-prompts

GitHub 체크포인트: Phase 종료 직후 테스트 결과를 확인한 다음 commit/push 한다. main 에 병합되는 마일스톤이라면 태그도 push 하는 것을 권장한다.

Phase 5 - Executor

항목	내용
구현 내용	async 실행, retry/timeout, rate limit, 세션, 원문 로그, token/latency
완료 조건	Fake target 에 동시 요청 및 실패/재시도 케이스 통과
권장 Commit	feat: add asynchronous attack executor
권장 Tag	v0.5.0-executor

GitHub 체크포인트: Phase 종료 직후 테스트 결과를 확인한 다음 commit/push 한다. main 에 병합되는 마일스톤이라면 태그도 push 하는 것을 권장한다.

Phase 6 - Judge

항목	내용
구현 내용	Rule → Regex → Canary → Semantic → LLM → Ensemble
완료 조건	정상/취약 fixture 에 대한 기대 판정 정확도 확보
권장 Commit	feat: implement multi-signal vulnerability judging
권장 Tag	v0.6.0-judge

GitHub 체크포인트: Phase 종료 직후 테스트 결과를 확인한 다음 commit/push 한다. main 에 병합되는 마일스톤이라면 태그도 push 하는 것을 권장한다.

Phase 7 - Reproducer

항목	내용
구현 내용	반복 실행, negative/positive control, variation, 재현률, 상태 전이
완료 조건	candidate 가 confirmed/rejected/unstable 로 안정적으로 분류
권장 Commit	feat: add finding reproduction and control testing

권장 Tag	v0.7.0-reproduce
--------	------------------

GitHub 체크포인트: Phase 종료 직후 테스트 결과를 확인한 다음 commit/push 한다. main 에 병합되는 마일스톤이라면 태그도 push 하는 것을 권장한다.

Phase 8 - Promptfoo + Garak

항목	내용
구현 내용	두 도구 Adapter, 출력 정규화, artifact 보존
완료 조건	두 Scanner 결과가 공통 NormalizedResult 로 변환
권장 Commit	feat: integrate promptfoo and garak scanners
권장 Tag	v0.8.0-llm-scanners

GitHub 체크포인트: Phase 종료 직후 테스트 결과를 확인한 다음 commit/push 한다. main 에 병합되는 마일스톤이라면 태그도 push 하는 것을 권장한다.

Phase 9 - Mutation Engine

항목	내용
구현 내용	paraphrase/translation/wrapping/multi-turn/social 전략, lineage
완료 조건	parent-child 계보와 중복 제거 및 전략별 통계 기록
권장 Commit	feat: add prompt mutation and lineage engine
권장 Tag	v0.9.0-mutation

GitHub 체크포인트: Phase 종료 직후 테스트 결과를 확인한 다음 commit/push 한다. main 에 병합되는 마일스톤이라면 태그도 push 하는 것을 권장한다.

Phase 10 - PyRIT Adaptive

항목	내용
구현 내용	PyRIT 연동, Candidate 기반 adaptive/multi-turn 후속 검증
완료 조건	PyRIT 결과도 동일 Reproducer/Judge 경로로 유입
권장 Commit	feat: integrate PyRIT adaptive red teaming
권장 Tag	v0.10.0-adaptive

GitHub 체크포인트: Phase 종료 직후 테스트 결과를 확인한 다음 commit/push 한다. main 에 병합되는 마일스톤이라면 태그도 push 하는 것을 권장한다.

Phase 11 - Recon

항목	내용
구현 내용	Subfinder → httpx → Katana → ffuf, Asset/Endpoint 저장
완료 조건	허가된 테스트 도메인에서 asset map 생성
권장 Commit	feat: add automated reconnaissance pipeline
권장 Tag	v0.11.0-recon

GitHub 체크포인트: Phase 종료 직후 테스트 결과를 확인한 다음 commit/push 한다. main 에 병합되는 마일스톤이라면 태그도 push 하는 것을 권장한다.

Phase 12 - Web Security Tools

항목	내용
구현 내용	Nuclei, Dalfox, TruffleHog Adapter
완료 조건	각 도구의 샘플 결과가 FindingCandidate 로 정규화
권장 Commit	feat: integrate web vulnerability and secret scanners
권장 Tag	v0.12.0-web-security

GitHub 체크포인트: Phase 종료 직후 테스트 결과를 확인한 다음 commit/push 한다. main 에 병합되는 마일스톤이라면 태그도 push 하는 것을 권장한다.

Phase 13 - AI Endpoint Classifier

항목	내용
구현 내용	chat/assistant/search/generate/API 패턴 + 콘텐츠 기반 분류
완료 조건	Endpoint 를 Normal API/AI API/Chat/RAG/Agent/Unknown 으로 분류
권장 Commit	feat: add AI endpoint classification
권장 Tag	v0.13.0-ai-discovery

GitHub 체크포인트: Phase 종료 직후 테스트 결과를 확인한 다음 commit/push 한다. main 에 병합되는 마일스톤이라면 태그도 push 하는 것을 권장한다.

Phase 14 - Dedup

항목	내용
구현 내용	hash/category/endpoint/signature/semantic 기반 중복 제거
완료 조건	동일 root cause 가 단일 Cluster 로 합쳐짐
권장 Commit	feat: implement finding deduplication
권장 Tag	v0.14.0-dedup

GitHub 체크포인트: Phase 종료 직후 테스트 결과를 확인한 다음 commit/push 한다. main 에 병합되는 마일스톤이라면 태그도 push 하는 것을 권장한다.

Phase 15 - Reporting

항목	내용
구현 내용	Markdown/JSON 보고서, Evidence 참조, 재현률/impact 자동 입력
완료 조건	Confirmed Finding 으로 재현 가능한 보고서 생성
권장 Commit	feat: add automated bug bounty report generator
권장 Tag	v0.15.0-reporting

GitHub 체크포인트: Phase 종료 직후 테스트 결과를 확인한 다음 commit/push 한다. main 에 병합되는 마일스톤이라면 태그도 push 하는 것을 권장한다.

Phase 16 - Orchestrator

항목	내용
구현 내용	scope → recon → classify → scan → judge → reproduce → dedup → report 연결
완료 조건	`scan --profile full` 이 테스트 환경에서 끝까지 실행
권장 Commit	feat: complete end-to-end scan orchestration
권장 Tag	v0.16.0-pipeline

GitHub 체크포인트: Phase 종료 직후 테스트 결과를 확인한 다음 commit/push 한다. main 에 병합되는 마일스톤이라면 태그도 push 하는 것을 권장한다.

Phase 17 - 안정화 / v1

항목	내용
구현 내용	unit/integration, resume, timeout, DB recovery, failure handling, 문서화
완료 조건	전체 테스트 green, clean install 에서 동일 결과, 주요 오류 graceful handling
권장 Commit	test: stabilize end-to-end vulnerability pipeline
권장 Tag	v1.0.0

GitHub 체크포인트: Phase 종료 직후 테스트 결과를 확인한 다음 commit/push 한다. main 에 병합되는 마일스톤이라면 태그도 push 하는 것을 권장한다.

18. GitHub 개발 운영 전략

혼자 개발하더라도 기능 브랜치와 의미 있는 커밋을 유지하는 편이 좋다. 커밋 히스토리는 코드 복구 수단일 뿐 아니라 설계와 구현 능력을 보여주는 포트폴리오 기록이 된다.

```
main
├─ develop
│   ├── feat/scope-engine
│   ├── feat/target-adapter
│   ├── feat/prompt-engine
│   ├── feat/judge
│   └── feat/reproducer
└─ feature branch
    -> unit/integration test
    -> develop merge
    -> phase completion review
    -> main merge
    -> tag
    -> push
```

- `feat:` 신규 기능, `fix:` 오류 수정, `refactor:` 구조 변경, `test:` 테스트, `docs:` 문서, `chore:` 환경/의존성으로 커밋 접두어를 통일한다.
- API key, token, raw private data, 실제 비밀값은 절대로 커밋하지 않는다. `.env.example`에는 변수명만 둔다.
- runs/, evidence/에 민감정보가 저장될 수 있으므로 기본 `.gitignore` 대상으로 둔다.
- 마일스톤마다 CHANGELOG 또는 GitHub Release note 를 남기면 구현 과정 추적성이 좋아진다.
- Phase 가 끝날 때마다 push 하는 것을 기본 권고사항으로 두고, 다음 Phase 를 시작하기 전에 원격 저장소 상태를 확인한다.

19. MVP 경계 및 우선순위

MVP 1

Scope → Target → Testcase → Executor → Judge → Reproducer → Evidence → Report 까지만 먼저 완성한다. 자동 Recon 과 10 개 외부 도구 전체 통합은 이후 Phase 로 넘긴다.

MVP 1에서는 사용자가 직접 테스트 대상 API 를 지정해도 충분하다. 이 단계에서 “발견”보다 “검증 가능한 Finding 생성” 품질을 먼저 확보해야 한다.

```
MVP 1
Scope
-> Target Adapter
-> Testcase
-> Executor
-> Judge
-> Reproducer
-> Evidence
-> Report

MVP 2
Promptfoo + Garak + Mutation + PyRIT

MVP 3
Recon + Web Security Tools + AI Endpoint Classifier

v1
Dedup + Full Orchestrator + Stability
```

20. 테스트 전략

테스트 계층

주요 검증

Unit	schema, scope matcher, parser, judge, hash, lineage, dedup 규칙
Fixture	정상/취약 LLM 응답 샘플, 도구별 JSON/JSONL sample
Integration	Fake LLM + Executor + Judge + Reproducer + DB
Tool Integration	외부 CLI version/exit code/output parser
E2E	허가된 로컬 lab 에서 full profile 실행
Regression	과거 Confirmed Finding 이 버전 변경 후에도 재현되는지
Safety	out-of-scope redirect, request budget, concurrency enforcement

20.1 Phase 완료 Gate

1. formatter / lint
2. unit tests
3. relevant integration tests
4. sample run
5. evidence check
6. README / config example update
7. git diff review
8. commit
9. push
10. milestone 이면 tag + push

21. v2 확장 방향

- PostgreSQL + Redis + Worker 기반 분산 스캔
- React/Vite Dashboard 및 Finding triage UI
- pgvector 기반 Finding/Mutation semantic clustering
- 과거 성공 Mutation 검색을 이용한 Attack Planner
- Continuous monitoring / scheduled regression red team
- GitHub Actions 등 CI/CD 에서 LLM security regression 실행
- 프로그램별 버그바운티 정책과 허용 테스트 유형을 Policy Profile 로 관리
- Tool/MCP 권한 그래프 및 Agent 행동 감사 로그
- 비용·토큰·성공률을 함께 최적화하는 실험 스케줄러

22. 주요 위험과 대응

위험	대응
False Positive	Canary/Rule/Control/Reproducer 를 우선하고 LLM Judge 는 보조로 사용
범위 이탈	Scope Engine 을 네트워크 게이트로 강제, redirect 재검증
비용 폭증	Run budget, token limit, mutation cap, early stopping
도구 출력 변경	Adapter fixture 및 parser regression test
Prompt 중복 폭증	hash dedup + lineage cap + strategy budget
민감 Evidence 노출	로컬 암호화/권한, gitignore, report redaction
재현 불가	model/version/temp/session/timestamp 를 모두 기록
툴 종속성	Core 가 외부 스캐너 타입을 직접 참조하지 않도록 NormalizedResult 사용

23. 최종 구현 체크리스트

- Scope 검증 없이는 외부 요청이 실행되지 않는가?
- 각 외부 Tool 이 Adapter 를 통해 공통 결과 형식으로 들어오는가?
- Prompt 와 Mutation 의 source/parent/hash 가 남는가?
- Request/Response 원문과 Judge 근거가 Evidence 에 남는가?
- Candidate 를 자동으로 Confirmed 로 단정하지 않고 Reproducer 를 거치는가?
- Control Test 와 reproduction rate 가 보고서에 포함되는가?
- 동일 root cause 중복 Finding 이 합쳐지는가?
- Report 가 사람이 다시 실행할 수 있는 재현 절차를 제공하는가?

- 민감 파일이 `.gitignore`에 포함되어 있는가?
- 각 Phase 종료 시 test → commit → push 가 수행되었는가?
- 의미 있는 마일스톤마다 태그가 원격 GitHub에 존재하는가?

24. 바로 시작할 작업 순서

6. GitHub Repository 생성 및 Phase 0 구조를 commit/push 한다.
7. Core Data Model 과 SQLite schema 를 구현하고 v0.1.0-core 를 만든다.
8. Scope Engine 을 구현해 이후 모든 네트워크 모듈의 공통 게이트로 만든다.
9. Fake LLM 과 Target Adapter 를 구현해 외부 서비스 없이 통합 테스트 가능한 환경을 만든다.
10. YAML Testcase/Prompt Engine 과 교재 기반 Seed 를 최소 세트로 등록한다.
11. Executor → Judge → Reproducer → Evidence → Report 순으로 MVP 1 을 완성한다.
12. MVP 1 을 GitHub Release 로 남긴 뒤 Promptfoo/Garak 를 통합한다.
13. Mutation Engine 과 PyRIT 을 붙여 adaptive testing 을 추가한다.
14. Recon/Web Tool 을 붙인 뒤 AI Endpoint Classifier 와 전체 Orchestrator 를 구현한다.
15. 마지막으로 Dedup, 안정화, 문서화 후 v1.0.0 을 태깅한다.

권장 개발 습관

구현 단위를 작게 유지한다. “이번 주에 전체 자동화”보다 “이번 Phase 에서 완료 조건을 충족하고 GitHub 에 복구 가능한 상태로 남긴다”를 기준으로 진행하는 편이 훨씬 안정적이다.

부록 A. 권장 `tools.yaml` 개념

```
tools:
  promptfoo:
    enabled: true
    category: llm_redteam
    adapter: adapters.llm.promptfoo
    output: json

  garak:
    enabled: true
    category: llm_scanner
    adapter: adapters.llm.garak
    output: jsonl

  pyrit:
    enabled: true
    category: llm_redteam
    adapter: adapters.llm.pyrit

  subfinder: { enabled: true, adapter: adapters.recon.subfinder }
  httpx:      { enabled: true, adapter: adapters.recon.httpx }
  katana:     { enabled: true, adapter: adapters.discovery.katana }
  ffuf:       { enabled: true, adapter: adapters.discovery.ffuf }
  nuclei:     { enabled: true, adapter: adapters.scanner.nuclei }
  dalfox:     { enabled: true, adapter: adapters.scanner.dalfox }
  trufflehog: { enabled: true, adapter: adapters.secrets.trufflehog }
```

부록 B. 권장 버전 태그 흐름

v0.0.1	Repository initialized
v0.1.0-core	Core models/storage
v0.2.0-scope	Scope engine
v0.3.0-targets	Target adapters
v0.4.0-prompts	Testcase/prompt engine
v0.5.0-executor	Async execution
v0.6.0-judge	Multi-signal judges
v0.7.0-reproduce	Reproduction/control
v0.8.0-llm-scanners	Promptfoo/Garak
v0.9.0-mutation	Mutation lineage
v0.10.0-adaptive	PyRIT
v0.11.0-recon	Recon pipeline
v0.12.0-web-security	Web scanners
v0.13.0-ai-discovery	Endpoint classifier

v0.14.0-dedup	Finding clustering
v0.15.0-reporting	Report generator
v0.16.0-pipeline	End-to-end orchestrator
v1.0.0	Stable first release